Gains Hide Failures: A Ledger Evaluation of a Commitment Guard for AAC Sentence Expansion

Anonymous submission

Abstract

People who use AAC (augmentative and alternative communication) pick a few words, and a language model turns them into a sentence the app speaks, sometimes dropping a "not". We evaluate a commitment *guard*, which makes the model rewrite when its sentence contradicts the user's input, in a *ledger* that sets each gain beside what it leaves broken, what it breaks and what it costs, against free baselines. On 78 fresh speech-act families the guard raises the pass rate (joint pass) by +9.83 points [8.39, 11.32] and repairs 52.51% of strict-check failures, with 0 strict-check pass-to-fail changes among 1,394 correct texts. Its feedback beats plain retrying by +6.20 points [4.75, 7.64] (guard at temperature 0, resampling at 0.7). The ledger shows what a gain-only report hides: the repair rate fell from 78.8% in development, 12.13% of targets still fail, and in a 40-text pilot repairs were usable in 58.8% of resolved items against 89.5% for controls. A 120-text audit, designed for 3 independent raters per text, tests this deployment clause under a rule frozen before any return. Models commit to the affirmative reading in 99.9% of committed answers even when told not to assume, so their rare errors are silent reversals. The ledger yields falsifiable predictions and a pre-specified human-effect design.

1 Introduction

People who use AAC often compose by selecting a few words or symbols, because every selection costs time and effort (Valencia et al. 2023; Cai et al. 2024). LLM keyword expansion turns `want go party` into a full sentence (Shen et al. 2022; Cai et al. 2024), which a system may speak on the user's behalf, sometimes after review (Hancock, Naaman, and Levy 2020; Valencia et al. 2020). But the model decides more than words: whether something happened or did not, whether the sentence asks or tells, whether it says "please". Given `not like medication`, a sentence that drops the negation, turns it into a question or adds politeness is fluent and wrong. Three kinds of fix are being explored: check the output against the user's words and regenerate on a conflict (Madaan et al. 2023; Gou et al. 2024), offer suggestions that span the readings the words leave open, or let the user add a one-tap cue such as `[NOT]` (Valencia et al. 2024; Mao et al. 2025; Xiang et al. 2025). Such work commonly reports gains of the accuracy or similarity kind: exact-expansion accuracy (Cai et al. 2022), a semantic-consistency rating with keystroke saving (Shen et al. 2022), entry-rate and motor savings (Cai et al. 2024). These are real gains, and some work also reports collateral change (Section 2). A gain-only report hides what a fix leaves unfixed, what it changes that was already right, and whether the repaired text is something a person could use.

User input	Model wrote	Guard output	What the scorer and the person see
`[?] [I] bring friend`	"Is it okay if I bring a friend?"	"Is [I] bring a friend?"	Scored a pass (it keeps the question label). Not a sentence.
`knock come`	"Please knock before you come in."	"Don't knock before you come in."	A menu meant to widen choices turned a polite request into a prohibition.
`[ASK] [YOU] [PLAIN] wear headphones`	"You wear headphones."	"Ask you to wear headphones."	Scored a pass. Not what was meant.

Figure 1: Illustrative failures found by the audit, real stored outputs. Bracketed tokens in an input are cue chips, single taps that fix one commitment: `[NOT]` negation, `[?]` a question, `[ASK]` and `[YOU]` an ask addressed to the listener, `[I]` the speaker as agent, `[PLAIN]` no added politeness. They are not typical: each is the first case in a fixed hash order within its category, and the rate of its category is given here. Row 1: the guard changed 26 of 1,394 confirmation texts that were already correct; none became a strict-check failure, but some changed texts, like this one, are not sentences. Row 2: menu rewrite; sets offering both a plain and a polite reading fell from 26/40 to 12/40. Row 3: second of the first two repairs in hash order (the first is a clean fix, Table 3); 4/63 scored repairs begin this way, a post hoc count.

Thesis and ledger. Where only gains are reported, failures stay hidden, so we report a ledger. Beside each benefit it

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

records *offline failure indicators* (events in stored text that a gain-only report omits: failures that remain, correct outputs the fix changed, readings a menu no longer offers, silent reversals under a default that is right on average), the cost, and where the intervention applies. None of these indicators is a measured harm to a person. Our running case is keyword-to-sentence expansion with four LLMs on minimal-pair item families, defined in Section 3.

Contributions. (1) A ledger and its design rules: attribution decides what an intervention may be credited with; a free baseline and pass lines fixed in advance decide whether it merits a human study (§5). (2) One lead case, a commitment guard, evaluated end to end: on a confirmation set with a frozen rule it raises the joint pass rate by +9.83 points, repairs 52.51% of strict-check failures with 0 strict-check pass-to-fail changes (26 of 1,394 already-correct texts were rewritten), and its feedback beats resampling by +6.20 points (+6.94 at matched temperature); the ledger adds the residual failures, the share a free baseline earns, and a usability pilot (§6). (3) Two more ledger rows (a menu and a cue), realistic-input checks with human-written pairs and human-chosen keywords, and an analysis of the affirmative default as a good guess with an asymmetric cost: models commit to it in 99.9% of committed answers even when told not to assume, so its rare errors are silent reversals. (4) A pre-specified human-effect design that turns the ledger into falsifiable predictions about people (§7). We ran no study with AAC users; every result is a change in stored text.

2 Related Work

LLMs in AAC. LLM expansion and suggestion systems for AAC include KWickChat (Shen et al. 2022) and SpeakFaster (Cai et al. 2024), and AAC users have described how such systems help and hinder them (Valencia et al. 2023). Several add user-side control over intent (Valencia et al. 2024; Mao et al. 2025; Xiang et al. 2025; Lacosse, Tano, and McNamee 2026); others measure whether affect or intent survives generation (Choudhury, Kumar, and Martin 2025). Frisch et al. argue that AI-powered AAC needs technical, user-mediated and functional evaluation together (Frisch et al. 2026); our ledger is a technical layer shaped so that a functional study can test it.

Editing, preservation and evaluation. Generate–critique–revise loops (Madaan et al. 2023; Gou et al. 2024) and conditioning a rewrite on the source dialog act (Qu, Ungar, and Sedoc 2023) are established, as is reporting a targeted change together with collateral change (Ziegenbein, Stahl, and Wachsmuth 2026; Elgaar and Amiri 2025). Style-transfer meta-evaluation shows that preservation metrics look valid only when test data do not confound the intended change with content errors (Pauli, Augenstein, and Assent 2025); models have default politeness strategies (Zhao and Hawkins 2025). We claim neither the loop nor gain–harm accounting. We add accounting conditioned on what the input *supplied*, against a free baseline, plus a human usability check on repaired text. Suggested text shifts what people write and believe (Arnold, Chauncey, and Gajos 2020; Jakesch et al. 2023; Hohenstein et al. 2023; Agarwal, Naaman, and Vashistha 2025); those are human-outcome studies of exposure, whereas ours is a pre-deployment study of interventions meant to *prevent* unwanted shifts. LLM judges (Zheng et al. 2023) can agree with each other more than with people on pragmatic labels (Wang, Sun, and Guo 2026; Panickssery, Bowman, and Feng 2024), which bounds what our machine labels can show.

3 Setting

Items and populations. An item is a minimal-pair family ("I don't like the new medication" / "I like the new medication"); each source sentence is shown to the model under five input conditions (Table 1). The *development set* has 58 constructed families on six dimensions (after quarantining two families whose scoring rule has a known blind spot); the guard was built and tested there, so its numbers are development results, and the menu and cue rows use it. The *confirmation set* has 240 fresh constructed families (polarity, speech act, politeness), 238 analyzed after a pre-generation exclusion, with three contrasts pre-specified in version control; a hash of the protocol was deposited publicly after the confirmation freeze, so it does not certify the freeze, and the repository timestamps are not independently verifiable. Its confirmation is qualified because 48 cached requests (12 distinct primary ones) predate the final item freeze. Its 78 speech-act families are the separate population on which the guard was rerun unchanged. The *human-written pairs* are 547 kept minimal pairs written by 20 non-author annotators. The *keyword study* has 20 annotators type their own keywords for 600 AACText messages (fictional messages written by crowd workers imagining AAC use, not messages of AAC users). The AAC input studied is word input only; symbol and grid-based AAC are out of scope.

Scoring and attribution. Four fixed models write each sentence: Qwen2.5-7B-Instruct, Llama-3.1-8B-Instruct and Gemma-3-4B-it (4-bit, llama.cpp) and one commercial API model (gpt-6.1-terra). An LLM extractor (two prompt variants) labels each output's dimensions without seeing the source, model or condition; a second-family arbiter resolves disagreements on critical labels; deterministic code turns labels into verdicts, identically for treated and untreated outputs. A target is one (item, model, input condition) outcome. Because the input condition fixes which values the model was given, a miss is attributed to one of three causes: *loss* (the input never carried the value), *violation* (it carried the value and the output changed it) or *assumption* (the input was ambiguous and the output committed anyway). This holds only as far as the extractor and the conditions are right. A checker can enforce only a value the user supplied, so the guard is credited with violations alone; when the keywords leave a dimension open, the remedies are to show alternatives (menu) or to let the user supply the cue. In the confirmation set, 259 disagreement rows (177 units) were resolved by a single arbiter after outcomes were known, blind to reference, model and condition; masking them as unknown gives the same three conclusions. Against two human annotators on 163 sampled untreated outputs, extractor labels agree at held-out $\kappa = 0.91$–0.97 on all 7 label fields (pre-specified gate 0.6;

Glossary			
Family	Two source sentences differing on one critical dimension, with the same bare keywords	*Cue, chip*	One tap that adds a commitment, such as `[NOT]`, `[?]`, `[PLAIN]`
Critical dimension	Polarity, agent, certainty, speech act (ask, tell, request), politeness, time/quantity	*Input conditions*	Keywords only; keywords plus a length-matched neutral word; plus the critical cue; plus all cues; the typed sentence ("typed input", the full channel)
Pass (joint pass)	Keeps the critical value and every other scored supplied predicate; the headline metric	*First pass*	The model's single greedy sentence, before any intervention
Strict-check pass	Keeps the critical value, contradicts no supplied dimension, adds no politeness; a failure is a *strict-check failure*, and *repair rate* is the share of these that the intervention fixes	*Strip-only*	Free baseline: delete an unrequested "please", "pls" or "plz" (no model call)
Violation	The input carried a value and the output changed it	*Offline failure indicator*	Event in stored text that a gain-only report omits

Table 1: Glossary. All scores are partial predicates computed from machine-extracted labels, not judgments of usability.

per-field n is smaller because ties and unsure items are excluded). Counting ties and unresolved items as disagreement instead, the critical-dimension κ is 0.925–0.931 across the three human gates, and added content is the weakest field (0.692–0.890). That check covers untreated outputs only; it does not validate repaired or rewritten text, and the pilot audit (Section 6.2) gives the only human check of repairs. The item construction and scorer are described in an anonymous concurrent submission (Anonymous 2027), which shares data with this paper (the confirmation items, the human-written pairs and the scorer) and reports the checker's own tests; this paper's guard and ledger results are reported here in full, and the description above is what they use.

4 Interventions

The commitment guard (lead; system side). After the first pass, a deterministic lexical monitor (regexes and lexicons for negation, person, modality, question form, numbers, tense and "please") compares the output with the values in the visible input: the cues, or for typed input the monitor's own labels of the typed sentence. It also flags a politeness marker that was not supplied. On a flag, the same conversation receives one feedback turn ("Your sentence must keep: negated; no added politeness. Write the sentence again with the same meaning otherwise.") and is regenerated, up to two retries. The guard returns the first unflagged attempt, else the attempt with fewest flags, ties going to the original. The strip-only baseline covers one failure type (an added "please"); it is a floor for a trivial rule, not a competitor that handles negation, person or sentence type; the second free baseline is plain retrying. The no-feedback baseline (G1r) keeps the monitor, trigger and retry cap but regenerates from the original prompt with no feedback turn, at temperature 0.7 because a temperature-0 retry repeats the first pass; the guard at the same temperature (G1t) isolates the feedback message.

The menu (secondary; keywords only). Starting from a baseline menu of three suggestions, the complete menu keeps the top suggestion and, for polarity then certainty, rewrites the third (then second) slot into a missing reading with one model call, checked by the monitor with one feedback retry. It was chosen over a variant on a 12-family pilot before the test run.

The cue (secondary; user side). The user adds one commitment cue, such as `[NOT]`, `[MAYBE]`, `[?]` or `[PLAIN]`. We treat it as a *probe* of whether information the user supplies is used, not as a proposed interface. Its real cost depends on the access method (one dwell in eye gaze, a menu traversal in scanning, a fixed button in a symbol grid), and it adds a vocabulary to learn, a decision for every message, and a risk that cues are copied literally into the output; none of these is measured here.

5 Evaluation Design

Paired comparisons of stored configurations. For system-side interventions, the untreated outcome is the *stored* first-pass output for the same request, and the treated outcome is what the intervention returns from it. This removes between-request variation but does not identify an effect on people. Pairing matters because the same prompt does not give the same text twice: reissuing a request reproduced the text in only 76.4–90.8% of local and 44.4–71.4% of API requests. Estimands are row-weighted means over targets; the four models are fixed, not sampled. Intervals are 95% percentile intervals from whole-family or cluster resamples (2,000 on development, 10,000 elsewhere; fixed seeds); they are pointwise and do not correct for exploratory selection or label error. Thousands of rows are not thousands of independent tests: the guard's 63/80 development repairs come from 26 families.

Offline failure indicators and cost. *Changed-correct* counts already-correct outputs whose text the intervention changed; *correct→failure* is the share that became scored failures; *residual* failures remain after the intervention. We also show what a label cannot see, such as scored passes that are fragments, and, for repaired text, a human usability rating. Cost is extra model calls and latency (local latency is not a deployment prediction). Pass lines are coarse by design: they decide whether an intervention merits a human study.

	Guard (lead)	**Menu (bounded)**	**Cue (bounded)**
Defect targeted	Violation of a supplied value	Assumption under ambiguous input	Loss of a value at input
Comparison	Stored first pass for the same request; strip-only (0 calls)	Stored baseline menu, same request	Length-matched neutral word
Population	Development: 1,392 cue targets; confirmation: 1,872 speech-act targets	Development, keywords-only sets of three	Development: 464 targets; confirmation speech act: 624 targets
Primary benefit	Strict-check pass; pass (joint) on confirmation	Sets offering both readings	Critical value kept
Offline failure indicators	Failures that remain; correct text changed; passes that are fragments; *human-rated usability of repairs*	Coverage lost on other dimensions; rewrites changing sentence type	Matches lost; misses unrepaired; gap to typed input; silent reversals when no cue is typed
Applicability	Supplied cues only	Ambiguous input only	Dimension-specific
Cost	Model calls, latency	Model calls, latency	Selections (access-dependent)
Pass line (fixed in advance)	Repair $\geq$30% of failures *and* change $\leq$10% of correct outputs; confirmation pass-rate gain with lower bound > 0; deployment clause: usable $\geq$ control -10 points (Section 6.2)	Gain $\geq$30 points over baseline menu	Gain $\geq$10 points *and* $\geq$80% of typed-input rate

Table 2: Evaluation card: intervention, comparison, population, outcomes. Verdicts are in Table 3, Fig. 2 and Section 6.2. Development pass lines were fixed in the analysis code and checked on pilot data before the run, without external timestamps; the confirmation criteria were frozen in a hashed file before any retry request.

6 Results

6.1 The guard: about half of failures repaired, and feedback beats plain retrying

On the development cue conditions, strict-check pass rose from 94.25% to 98.78% of 1,392 targets (Table 3). Of the 80 initial failures the guard repaired 63 (78.8% [68.4, 88.9]), far above its 30% pass line. A gain-only report stops there. Deleting an unrequested "please" at zero calls already reaches 1,346 strict-check passes, so regeneration's increment is 2.08 points [1.15, 3.16]: 31 targets pass only with the guard and 2 only with deletion. This contrast alone does not isolate the feedback message; the no-feedback baseline below does. 17 of 80 failures remained, 12 of them involving speech act, and 14/17 were still flagged after both retries: detected, not repaired. The guard changed the text of 49/2,154 already-correct outputs (all inputs), none into a strict-check failure; yet "Would you mind repeating the last question" became "Please repeat the last question", still polite by the label and no longer what was typed. It flagged 157/1,856 inputs and added 0.104 model calls per input. Regenerating without feedback repairs little: it repaired 15/80 development failures (18.8%) against 63/80 with the guard, and 87/478 confirmation failures (18.2%) against 251/478. Feedback adds value beyond trying again: the guard's pass-rate increment over resampling (unmatched decoding) is +3.23 [1.87, 4.81] points in development and +6.20 [4.75, 7.64] in confirmation. The matched-decoding contrast, in which the guard also runs at temperature 0.7 (G1t $-$ G1r), is the clean feedback effect: +2.95 in development and +6.94 [5.34, 8.60] in confirmation. A retry often reproduced the first text (111/170 and 351/508 flagged requests), so these small models are nearly deterministic even at temperature 0.7; retries were held to the guard's budget. No arm turned a strict-check pass into a failure (0 and 0 cases).

Confirmation. Speech act was the guard's weakest dimension, so we reran the guard unchanged on the confirmation set's speech-act families (78 families, four models, three cue conditions, 1,872 targets). Criteria were frozen before any retry: at least 30% of strict failures repaired, at most 10% of strict-correct texts changed, and a joint-pass gain whose lower 95% bound exceeds zero. The pass rate rose from 78.79% to 88.62%, 9.83 points [8.39, 11.32]; the guard repaired 251/478 strict-check failures (52.51% [48.36, 56.70]), changed 26/1,394 already-correct texts and turned 0 of the strict-check passes into failures. Every scored criterion passed within the call caps. The repair rate fell from 78.8% in development to 52.51% on fresh items, so the development figure overstates it; a harder population and development tuning both fit, and we cannot separate them. Deletion alone gained 3.47 points, so the guard's increment over it is 6.36 points [5.29, 7.48], and 12.13% of targets remain strict-check failures. Of the 26 already-correct texts the guard rewrote, some are not sentences: in row 1 of Figure 1 the question label survives, so the text still scores as a pass.

6.2 What the pass does not show: repair usability

A pass checks a few labels, not a sentence. For `[ASK] [YOU] [PLAIN] wear headphones`, the guard turns "You wear headphones." into "Ask you to wear headphones.", which passes every scored predicate (Figure 1). We therefore had people judge repaired text. In a pilot audit, 14 reimbursed non-author raters judged 20 repaired outputs and 20 unchanged controls (outputs that had never failed) for usability, blind to which was which. By item consensus, repaired texts were usable in 10/17 resolved items (58.8%) against 17/19 (89.5%) for controls, a gap of -30.7 points [-54.7, -2.0], with 3 and 1 items unresolved. Counting unresolved items as unusable, the gap is -35.0 points [-57.4, -5.9]

	Development 58 families, 6 dimensions, 1,392 cue targets	**Confirmation (frozen)** 78 families, speech act, 1,872 targets
Benefit (% of targets)		
Strict-check pass: first pass / strip-only / guard	94.25 / 96.70 / 98.78	74.47 / 81.73 / 87.87
Pass (joint): same order	94.68 / 96.77 / 98.85	78.79 / 82.26 / 88.62
Guard − first pass, points	+4.53 strict-check	**+9.83** [8.39, 11.32] pass; +13.41 [11.38, 15.44] strict-check
Strict-check failures repaired (repair rate)	63/80 (78.8%)	251/478 (52.51%)
Free baseline: strip-only, zero calls		
Strip-only − first pass, points	+2.44 strict-check	+3.47 [2.78, 4.22] pass; +7.26 strict-check
Guard − strip-only, points	**+2.08** [1.15, 3.16] strict-check	+6.36 [5.29, 7.48] pass
Pass only with guard / only with strip-only	31 / 2	123 / 4
Free baseline 2: resample without feedback (G1r), T=0.7, same retry cap; G1t is the guard rerun at T=0.7		
Pass: resample G1r / guard G1t (T=0.7)	95.6 / 98.6	82.4 / 89.4
Repair rate: resample G1r	15/80 (18.8%)	87/478 (18.2%)
Guard (T=0) − G1r, points (pass)	**+3.23** [1.87, 4.81]	**+6.20** [4.75, 7.64]
G1t − G1r, matched decoding (pass)	+2.95 [1.65, 4.38]	+6.94 [5.34, 8.60]
What remains (offline failure indicators)		
Residual strict-check failures	17/1,392 (1.2%)	227/1,872 (12.13%)
Already-correct text changed	14/1,312 cue; 49/2,154 all inputs	26/1,394 (1.87%)
Strict-check pass → failure	0/1,312	0/1,394
Cost and verdict		
Extra calls per input	0.104 (1,856 inputs)	0.41 (1,848 inputs)
Pass line	passed	passed on the three scored criteria; usability clause open

Visible input	First pass	Guard output
Development set, repaired (Gemma, one cue; 1st by hash order; 63/80 scored failures repaired)		
`[PLAIN] let finish sentence`	"Please let me finish my sentence."	"Let me finish the sentence."
Development set, scored repair that is a fragment (Llama, all cues; 2nd by hash order; 4/63 scored repairs begin "Ask you", a post hoc lexical count, not a fluency rate)		
`[ASK] [YOU] [PLAIN] wear headphones`	"You wear headphones."	"Ask you to wear headphones."
Development set, failure that remains (Llama, all cues; 1st by hash order; 17/80 failures remain, 14/17 still flagged; the source asks whether the listener is charging it)		
`[?] [YOU] charge wheelchair`	"Can you charge my wheelchair?"	"Can you charge my wheelchair?"
Confirmation set, correct text changed, still a scored pass (Qwen, all cues; 1st by SHA-256 order; 26/1,394 strict-correct targets changed, none to a scored failure)		
`[?] [I] bring friend`	"Is it okay if I bring a friend?"	"Is [I] bring a friend?"

Table 3: The guard as one display. Left: benefit, the free baseline and what remains, on two *separate* populations that are never pooled; scored partial predicates, not usability. *Targets* are scored (item, model, input condition) outcomes; *inputs* are the distinct requests the guard processed. Development: 1,392 cue targets, plus 464 requests with no supplied cue (no strict target), give 1,856 inputs. Confirmation: 1,872 targets share 1,848 distinct requests. Pass means joint pass throughout; strict-check rows are labelled. Differences are computed from unrounded rates, so they can differ by 0.01 from the difference of the printed rates. Right: stored examples, verbatim, each following the rule named in its row; they are not typical. Brackets: 95% whole-family bootstrap intervals for paired differences. Repair rates condition on initial strict-check failures; already-correct and pass-to-failure rows on initial strict-check passes.

(50% against 85%). The pilot has three limits. (i) Raters' answers were highly redundant: their usable/unusable answers collapse to 2 distinct patterns (pairwise correlation 0.87), about 1.1 effective independent raters. (ii) Controls were

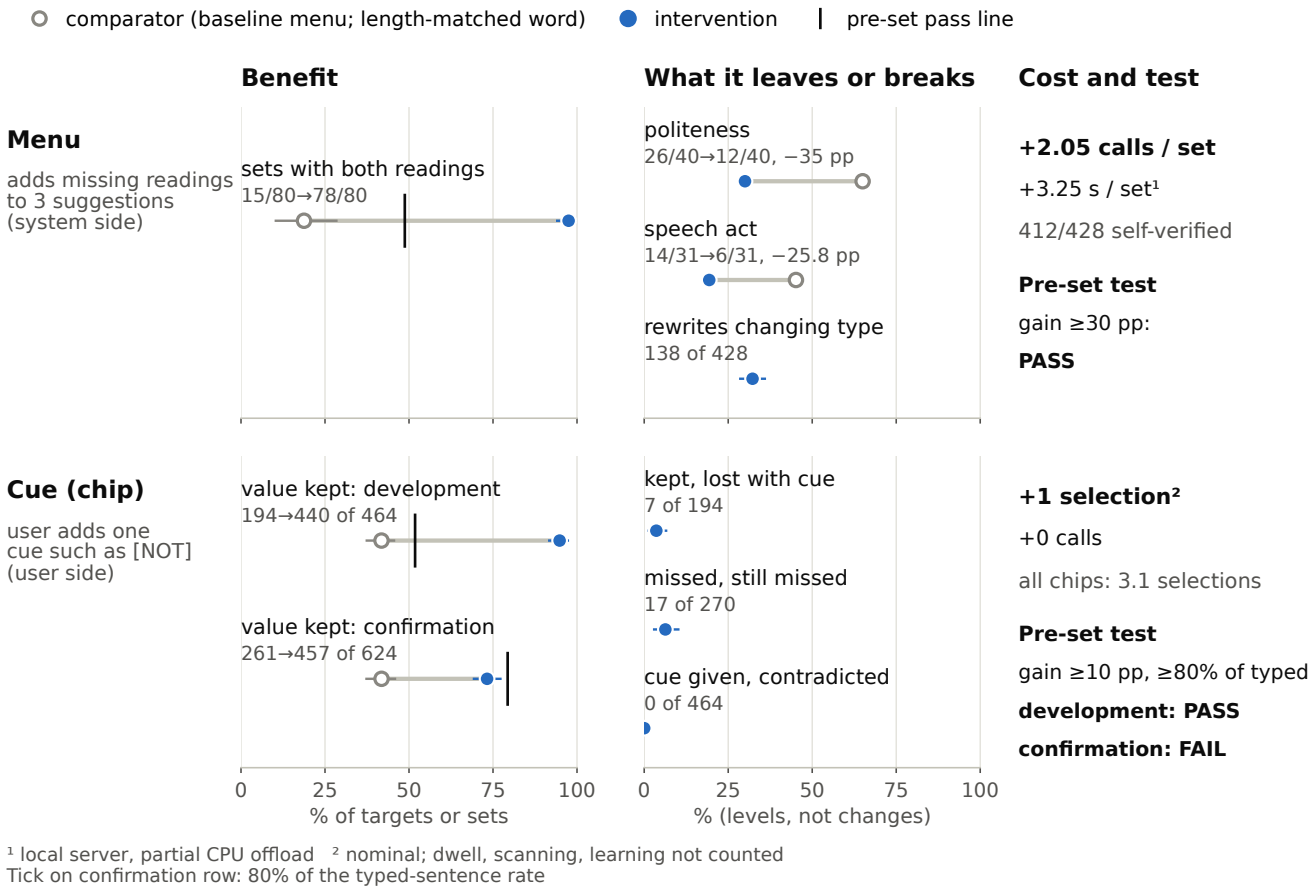


Figure 2: Two add-ons help in one place and cost something elsewhere (machine-extracted labels; four models). Open circle: comparator; filled: intervention; bars: 95% cluster-bootstrap intervals; black tick: the pass line set in advance; pp: percentage points. Plotted values are percent *levels*; changes in points are printed. The menu's politeness and speech-act rows come from families different from its target gain (40 and 31 sets). The cue passes both clauses on the development set but *fails* its ≥80%-of-typed clause on the confirmation set (only its gain clause replicates). The guard is in Table 3.

outcome-selected: all 20 passed the strict check at first pass, whereas 12/20 repaired items were known first-pass failures, so repaired and control items differ in difficulty. (iii) The dimension mix differs (politeness: 10 of 20 repaired against 2 of 20 controls; permutation $p = 0.031$). A second pool of the same design (confirmation-set outputs) showed no gap: 77.8% against 77.8%. Pooling the two pilot pools (post hoc) gives a gap of −15.2 points [−33.8, 4.5], which includes both zero and the −10-point line. Raters mostly agreed that the meaning of a scored-passing repair was kept (pilot: 14/16 resolved items; second pool: 16/19), so the problem is more often fluency and form (Table 4) than lost meaning: chip echoes, dropped content words and fragments, all passing the scored predicates. **The decisive test.** The pilot is a warning and the larger audit decides: a 120-text audit of new texts (half repaired, half never-failed controls, none from the pilots, designed for 3 independent raters per text, since the pilot's raters were effectively only about 1.1 independent, plus original-failed and strip-only arms to separate repair from item difficulty) has a decision rule frozen before any return: a pooled interval entirely below −10 points means the guard is killed for deployment and kept as a detector; one that includes −10 but excludes 0 means a cost of uncertain size; one that includes 0 means no detectable cost. The audit result is not yet in; it replaces the pilot as the primary estimate.

6.3 Two more ledger rows: the menu and the cue

Both rows show the pattern of the guard: a gain in one place, a cost or a failed clause in another. The complete menu raises coverage of both polarity or certainty readings, but on other dimensions it removes the polite option: for `knock come`, "Please knock before you come in." became "Don't knock before you come in." (Figure 1). A menu prompt that asks for different meanings also departs from a value the input already fixes (62.76% of baseline menus against 2.70% of sample sets for typed input), so a menu belongs on dimensions the input leaves open. The cue's gain over the length-matched word replicates on 78 fresh speech-act families (31.4 points, or 18.6 [10.0, 26.9] if "Can you. . . ?" counts as a question), but one cue reaches 73.8% of typed input, below its

Pool	Visible input	Stored repaired output
Pilot	`[ASK] [YOU] [PLAIN] speak slowly`	"Speak slowly to you."
	`[ASK] [YOU] [PLAIN] charge wheelchair`	"Ask you for the wheelchair charge."
	`evening take pills`	"Take pills evening."
Second	`[ASK] [YOU] [PLAIN] email manager`	"Ask you email manager."
	`[ASK] turn conditioner`	"Give me a turn conditioner."
	`[TELL] text plumber`	"The plumber."

Table 4: Repairs raters judged unusable by consensus, verbatim. Rule: the first three consensus-unusable repaired items in frozen hash order in each pool (pilot: 7/17 of rated repairs unusable; second pool: 4/18). A cue echo (a repair that begins "Ask you" or "Tell you") opens 1 of the three pilot examples and 1 of the three second-pool examples (3 and 1 of all unusable repairs in the pools).

	Gain	Cost or fresh-item check
Menu	Both readings offered in 78/80 sets against 15/80 (+78.75 points [70.00, 87.50]); passes its line	Plain and polite both offered: 26/40 to 12/40 (politeness families), 14/31 to 6/31 (speech-act families); 32.2% of 428 rewrites changed sentence type
Cue	Critical value kept: 41.8% to 94.8% (97.3% of typed input); passes both clauses	Fresh speech-act families: 41.83% to 73.24%, 73.8% of typed input (line 80%): *fails* one clause; 58/208 question targets became non-questions

Table 5: Two secondary ledger rows. Menu rows use development keyword-only sets; the politeness and speech-act sets are different families from the gain.

80% line, and question cues fail most visibly: `[?] parcel signature` became "Can you sign for this parcel?". The bare-keyword default is not a prompt artifact. On bare polarity inputs (4 models, both sources), affirmative output among committed answers was 99.5% with the cue legend, 98.4% with the legend removed and 99.9% when the prompt said markers may be missing and not to assume one (a second draw: 99.3, 98.6 and 99.9%). Models answered "unclear" in at most 0.7% of outputs even when told. You cannot prompt the default away, so the ledger must measure it.

6.4 Realistic-input checks

Constructed families may not look like what people write, so we ran the pipeline on 547 minimal pairs written by 20 non-author annotators and on keywords that 20 annotators typed for 600 AACText messages. On the human-written pairs the cue gain replicates on all 6 dimensions (35.3 to 82.7 points), and the three confirmation contrasts replicate in direction: speech-act cue +35.3 points, politeness addition +7.6, and a bare affirmative default of 99.8% [99.3, 100.0]. With 6–7 writers per contrast we claim direction and consistency: every writer's mean has the same sign, and writer-level intervals are [28.9, 41.7] and [3.7, 11.5] points for the first two (exact sign-flip one-sided $p = 0.016$, 0.016 and 0.008; smallest attainable 0.0078); the replication is exploratory. In the keyword study about one keyword set in five lost the negation (20.8% [16.9, 25.2]; 21.4% [15.4, 27.8] after resampling messages and reweighting to the sampling frame), on cue-enriched messages that carry a negation; a rule-based four-word compressor lost it 100.0% of the time, and people lost the polite cue more often (35.7%) than the compressor (17.3%).

Defaults: a good guess with an asymmetric cost. When keywords carry no negation word, the models write an affirmative sentence for 99.9% of such inputs. That is the Bayes-preferred guess: 8.6% of AACText messages are negated, and among human keyword sets with no negation word the source was negated only 2.7% [1.8, 3.7] of the time (2.0% from corpus prevalence), against 11.4% for the crude compressor; guessing affirmative is right 97.3% of the time. The harm is the asymmetric cost: about 24 [15, 33] of every 1,000 messages are silently reversed when the user types keywords unaided, against 0.8 spurious negations, and the guess breaks even only if a reversal costs under 36 times a spurious negation. For politeness the models match the preferred guess (they add polite wording to 6.7% of no-cue inputs against a posterior of 6.5%); for speech act "statement" is not preferred (the source was a question or request 47.9% of the time, models asserted one 28.2%), and they over-commit to it. The finding is not that models are biased: an average-accuracy metric rewards exactly the behavior that silently reverses meaning, so a ledger puts the cost of a default beside its hit rate. These figures rest on detector labels of source messages and one greedy draw per model.

7 From Ledger to a Pre-Specified Human-Effect Study

The ledger is a pre-deployment screen, and it earns its place if it predicts human outcomes. Each prediction below is falsifiable, comes with the study that tests it, and names what would demote it. The design is pre-specified in version control; the AAC-user study is the next step.

Predictions. H1 (writer intent). Among outputs the scorer counts as preserving a writer's critical value, writers report that the output "said what I meant" more often than among outputs it counts as failing; a gap whose interval includes zero demotes the scored endpoint. The first test is the *writer-intent check*: each of the 20 human-pair writers

judges 10 outputs generated from their own pairs (five with keywords only, five with one cue), choosing between their two source readings, "something else" or "unclear". Its result is not yet in. **H2 (repair usability).** A scored repair is usable at no less than 10 points below a never-failed control; the 120-text audit tests this under the frozen rule of Section 6.2. **H3 (collateral).** Offline changed-correct and coverage loss predict a later drop in users' "said what I meant" rate; if not, evaluation of this kind should be demoted. H1 and H2 are testable now with writers and raters; H3 needs AAC users.

Study for H3: a micro-randomized trial. *Estimand.* For adult AAC users who compose with LLM keyword expansion, the average causal excursion effect (Boruvka et al. 2018) on a proximal outcome of delivering the regenerated sentence versus the original, at decision points where the monitor flags a conflict, averaged over messages and time within user. *Arms and unit.* Randomization is per flagged message within user (Klasnja et al. 2015), with fixed probabilities, among three arms: silent regeneration, flag-and-confirm (the user sees the flag and chooses), and the original. The guard's detection value motivates the confirm arm. Cues can only be offered, an encouragement design with its own estimand, so they enter as a separate between-user factor, not randomized per message. *Outcomes.* Proximal: sent without edit, one-tap "said what I meant" on a random subset of sends (not every send, to limit burden), time to send. Longitudinal: weekly rates of both, abandonment of the feature, and reliance (whether acceptance of flagged text rises over weeks), with week as a pre-specified moderator. *Sample-size reasoning.* The monitor flagged about 8.5% of inputs on constructed items (the rate in real AAC use is unknown), so a small share of messages are randomized: 40 randomized decision points per user need roughly 473 messages, which is weeks of use. The "said what I meant" outcome is collected on a random 50% of sends, so only about 20 of those points per user carry it. Assuming a standardized proximal effect of 0.2, within-user correlation 0.3 and 80% power at the 5% level (typed assumptions, to be replaced by estimates from a first small pilot), about 132 users are needed for that outcome (125 if every send were rated). *Analysis.* Weighted and centered least squares on the pre-specified proximal outcome with user-clustered errors; moderators and the longitudinal contrasts are reported with intervals and without selection. *What the human audits can do.* The audit's 60 versus 60 texts detect a gap of about 16 points when the control rate is near 89%; the writer-intent check has 200 judgments, an effective 71 at an assumed within-writer correlation of 0.2, enough to detect a gap of about 23 points in the worst case. Both can demote the ledger's endpoints; the trial tests effects on people.

8 Limitations

Human outcomes. The results are changes in stored text; communication success, agency, effort and trust are untested, which is what Section 7 addresses. *Repair audit.* The pilot has 20 + 20 items, about 1.1 effective independent raters and outcome-selected controls; the 120-text audit is the primary estimate. *Machine labels.* Endpoints use an LLM extractor and deterministic verdicts, validated by two human annotators on untreated outputs; guard repairs and menu rewrites are checked only by the usability audit. *Baselines.* Strip-only covers one failure type; the resample baseline uses one retry seed per attempt and temperature 0.7 against the guard's 0, which the matched-decoding contrast addresses, and is exploratory. *Scope.* The guard was developed on the development set and confirmed on a separate one, whose status is qualified by early cached requests and post-outcome arbitration; the menu was run on neither the confirmation set nor supplied input; the human-written pairs have few writers. *Constructed items, fixed models.* Four fixed models describe these systems, not other models; AACText is crowd-written fiction, word input is the only modality, and many AAC users compose by rewriting or choosing among options, which we do not model. Examples follow declared, outcome-conditioned rules and illustrate rather than sample. *Community involvement.* No AAC user or autistic adviser contributed to the design or analysis; we draw on guidance such as AASPIRE (Nicolaidis et al. 2019) and on the double-empathy (Milton 2012) and heterogeneity (Zisk and Dalton 2019) cautions, assume no "autistic style", and limit claims to commitments explicit in the text; claims about benefit need AAC users and advisers. *Ethics.* No data were collected from AAC users. All annotators were adult non-authors recruited from the authors' university peers, identified only by an ID; each consented and was reimbursed for their time. They labelled model outputs and audited items under a fixed codebook, blinded to model, condition and source; some also chose keywords for public AACText messages or wrote new sentence pairs, released under CC BY 4.0 with recorded consent. No personal or health data were collected, and we did not seek an institutional review determination.

9 Conclusion

On fresh speech-act confirmation families the guard raises the pass rate by +9.83 points, repairs 52.51% of strict-check failures with 0 strict-check pass-to-fail changes, and its feedback beats resampling (+6.94 points at matched temperature). The ledger places beside that gain what a gain-only report omits: deletion alone earns +3.47 of the +9.83 points, the repair rate fell from development to confirmation, scored passes can be fragments, a pilot points to a usability cost which the frozen-rule audit will test, and a default right on average still reverses meaning silently at a rate accuracy cannot show. The menu and the cue show the same pattern: a gain in one place, a loss or a failed clause in another. The pre-specified study turns these offline findings into predictions that people, above all people who use AAC, can confirm or falsify. Analysis code, stored outputs and scripts that rebuild every number and figure accompany the submission as anonymized supplementary material.

References

Agarwal, D.; Naaman, M.; and Vashistha, A. 2025. AI Suggestions Homogenize Writing Toward Western Styles and Diminish Cultural Nuances. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. ACM.

Anonymous. 2027. Testing What an AAC Expander Is Allowed to Say: Input-Conditioned Commitment Contracts for LLM Sentence Expansion. Anonymized for review.

Arnold, K. C.; Chauncey, K.; and Gajos, K. Z. 2020. Predictive Text Encourages Predictable Writing. In *Proceedings of the 25th International Conference on Intelligent User Interfaces*. ACM.

Boruvka, A.; Almirall, D.; Witkiewitz, K.; and Murphy, S. A. 2018. Assessing Time-Varying Causal Effect Moderation in Mobile Health. *Journal of the American Statistical Association*, 113(523): 1112–1121.

Cai, S.; Venugopalan, S.; Seaver, K.; Xiao, X.; Tomanek, K.; Morris, M. R.; Kane, S.; Narayanan, A.; MacDonald, R. L.; Kornman, E.; Vance, D.; Casey, B.; Gleason, S. M.; Nelson, P. Q.; and Brenner, M. P. 2024. Using large language models to accelerate communication for eye gaze typing users with ALS. *Nature Communications*, 15: 9449.

Cai, S.; Venugopalan, S.; Tomanek, K.; Narayanan, A.; Morris, M. R.; and Brenner, M. P. 2022. Context-Aware Abbreviation Expansion Using Large Language Models. In *Proceedings of NAACL*, 1261–1275.

Choudhury, S. H.; Kumar, A.; and Martin, L. J. 2025. Evaluating Human-LLM Representation Alignment: A Case Study on Affective Sentence Generation for Augmentative and Alternative Communication. In *Proceedings of the 14th International Joint Conference on Natural Language Processing and the 4th Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics*, 1618–1637. The Asian Federation of Natural Language Processing and The Association for Computational Linguistics.

Elgaar, M.; and Amiri, H. 2025. Linguistically-Controlled Paraphrase Generation. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, 20842–20864. Association for Computational Linguistics.

Frisch, B.; Wade, W.; Gaines, D.; Kinsella, M.; Peters, B.; Broderick, T.; and Vertanen, K. 2026. It's Complicated: On the Design and Evaluation of AI-Powered AAC Interfaces. Position paper, Speech AI for All workshop at CHI 2026, arXiv:2606.24854.

Gou, Z.; Shao, Z.; Gong, Y.; Shen, Y.; Yang, Y.; Duan, N.; and Chen, W. 2024. CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing. In *International Conference on Learning Representations*.

Hancock, J. T.; Naaman, M.; and Levy, K. 2020. AI-Mediated Communication: Definition, Research Agenda, and Ethical Considerations. *Journal of Computer-Mediated Communication*, 25(1): 89–100.

Hohenstein, J.; Kizilcec, R. F.; DiFranzo, D.; Aghajari, Z.; Mieczkowski, H.; Levy, K.; Naaman, M.; Hancock, J.; and Jung, M. F. 2023. Artificial intelligence in communication impacts language and social relationships. *Scientific Reports*, 13: 5487.

Jakesch, M.; Bhat, A.; Buschek, D.; Zalmanson, L.; and Naaman, M. 2023. Co-Writing with Opinionated Language Models Affects Users' Views. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. ACM.

Klasnja, P.; Hekler, E. B.; Shiffman, S.; Boruvka, A.; Almirall, D.; Tewari, A.; and Murphy, S. A. 2015. Microrandomized Trials: An Experimental Design for Developing Just-in-Time Adaptive Interventions. *Health Psychology*, 34(Suppl.): 1220–1228.

Lacosse, E.; Tano, P.; and McNamee, D. C. 2026. Multi-System Cognitive Steering for Generative Communication. ArXiv preprint, version 1, October 2026, arXiv:2610.19177.

Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegreffe, S.; Alon, U.; Dziri, N.; Prabhumoye, S.; Yang, Y.; Gupta, S.; Majumder, B. P.; Hermann, K.; Welleck, S.; Yazdanbakhsh, A.; and Clark, P. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*, volume 36.

Mao, L.; Lee, J. H.; Faroqi-Shah, Y.; and Valencia, S. 2025. Design Probes for AI-Driven AAC: Addressing Complex Communication Needs in Aphasia. In *Proceedings of the 2025 ACM Designing Interactive Systems Conference*, 404–422. ACM.

Milton, D. E. M. 2012. On the Ontological Status of Autism: The 'Double Empathy Problem'. *Disability & Society*, 27(6): 883–887.

Nicolaidis, C.; Raymaker, D.; Kapp, S. K.; Baggs, A.; Ashkenazy, E.; McDonald, K.; Weiner, M.; Maslak, J.; Hunter, M.; and Joyce, A. 2019. The AASPIRE Practice-Based Guidelines for the Inclusion of Autistic Adults in Research as Co-Researchers and Study Participants. *Autism*, 23(8): 2007–2019.

Panickssery, A.; Bowman, S. R.; and Feng, S. 2024. LLM Evaluators Recognize and Favor Their Own Generations. In *Advances in Neural Information Processing Systems*, volume 37.

Pauli, A. B.; Augenstein, I.; and Assent, I. 2025. Mind the Style Gap: Meta-Evaluation of Style and Attribute Transfer Metrics. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, 21550–21564. Association for Computational Linguistics.

Qu, R.; Ungar, L.; and Sedoc, J. 2023. Conditioning on Dialog Acts improves Empathy Style Transfer. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 13254–13271. Association for Computational Linguistics.

Shen, J.; Yang, B.; Dudley, J.; and Kristensson, P. O. 2022. KWickChat: A Multi-Turn Dialogue System for AAC Using Context-Aware Sentence Generation by Bag-of-Keywords. In *Proceedings of the 27th International Conference on Intelligent User Interfaces*, 853–867. ACM.

Valencia, S.; Cave, R.; Kallarackal, K.; Seaver, K.; Terry, M.; and Kane, S. K. 2023. "The less I type, the better": How AI Language Models can Enhance or Impede Communication for AAC Users. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. ACM.

Valencia, S.; Huynh, J.; Jiang, E. Y.; Wu, Y.; Wan, T.; Zheng, Z.; Admoni, H.; Bigham, J. P.; and Pavel, A. 2024. COMPA: Using Conversation Context to Achieve Common Ground in AAC. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. ACM.

Valencia, S.; Pavel, A.; Santa Maria, J.; Yu, S.; Bigham, J. P.; and Admoni, H. 2020. Conversational Agency in Augmentative and Alternative Communication. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*. ACM.

Wang, R.; Sun, K.; and Guo, Y. 2026. Polite but Misaligned: Evaluating LLM Politeness Judgments Against Human Pragmatic Norms. In *Findings of the Association for Computational Linguistics: EMNLP 2026*, 26402–26421. Association for Computational Linguistics.

Xiang, W.; Xu, Y.; Fang, Y.; Teng, Z.; Jiang, Z.; Hu, B.; and Yang, J. 2025. SocializeChat: A GPT-Based AAC Tool Grounded in Personal Memories to Support Social Communication. ArXiv preprint, arXiv:2510.19017.

Zhao, H.; and Hawkins, R. D. 2025. Comparing human and LLM politeness strategies in free production. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 16188–16216. Association for Computational Linguistics.

Zheng, L.; Chiang, W.-L.; Sheng, Y.; Zhuang, S.; Wu, Z.; Zhuang, Y.; Lin, Z.; Li, Z.; Li, D.; Xing, E. P.; Zhang, H.; Gonzalez, J. E.; and Stoica, I. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems*, volume 36.

Ziegenbein, T.; Stahl, M.; and Wachsmuth, H. 2026. Teaching LLMs Human-Like Editing of Inappropriate Argumentation via Reinforcement Learning. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 38616–38637. Association for Computational Linguistics.

Zisk, A. H.; and Dalton, E. 2019. Augmentative and Alternative Communication for Speaking Autistic Adults: Overview and Recommendations. *Autism in Adulthood*, 1(2): 93–100.